SmartLearn — Progress Update

MILESTONE: 85% COMPLETED

C-Based Educational & Quiz Engine (Console Application)

PROJECT TEAM — GROUP 08

Abdur Rahman Jabir
ID: 2631530642

Ragib Sindid Priam
ID: 2631108042

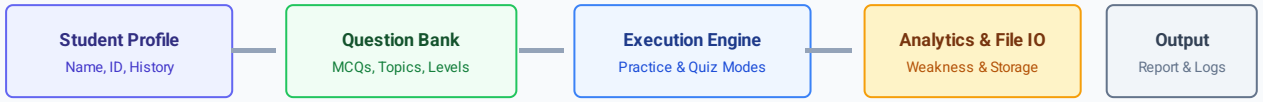
Md. Atikur Rahman
ID: 2632900042

Executive Summary: SmartLearn is an educational software developed in C to deliver structured topic-wise assessment and student analytics. The complete software architecture and program functions have been composed in the primary repository. Detailed functional specifications, data models, and real-life validation test suites are fully defined. Preliminary testing indicates stable operation of Practice/Quiz modes, scoring algorithms, and question randomization engines, with final refinements active in performance analytics UI and file storage routines.

85%	100%	95%	Active
OVERALL COMPLETION	REQUIREMENTS & DOCS	FUNCTION IMPLEMENTATION	REAL TEST SUITE

SYSTEM ARCHITECTURE & FUNCTION PIPELINE

GitHub: honu55/smartlearn-c-quiz-system



1. TECHNICAL MAPPING OF C CONCEPTS

SYSTEM MODULE	C CONCEPT IMPLEMENTATION	FUNCTIONAL REQUIREMENT STATUS
Profile Management	`struct Profile`, File I/O, Strings	COMPLETED Unique ID indexing & registration.
Question Bank System	Struct Arrays, Pointer Pass	COMPLETED Stores options, difficulty, keys.
Quiz Engine	`rand()`, `srand()`, Loops	COMPLETED Non-repeating random quiz generation.
Topic Analytics	Formulas, Conditionals	REFINING Strength & weakness topic tracking.
Data Persistence	`fopen()`, `fwrite()`, File IO	REFINING Persistent binary & text file save.

2. TEAM CONTRIBUTION

TEAM MEMBER	STUDENT ID	CONTRIBUTION
Abdur Rahman Jabir	2631530642	Quiz Engine, GitHub
Ragib Sindid Priam	2631108042	Question Bank
Md. Atikur Rahman	2632900042	Testing & Documentation

3. FUNCTION & MODULE IMPLEMENTATION MATRIX

FUNCTION / MODULE NAME	STATUS	FUNCTIONAL & TESTING DETAILS
`createStudentProfile()`	COMPLETE	Validates student name and ID input with unique string checks.
`selectSubjectAndTopic()`	COMPLETE	Provides dynamic menu options for subjects and sub-topics.
`loadQuestionBank()`	COMPLETE	Loads MCQ option structures, difficulty flags, and answer keys.
`runPracticeMode()`	COMPLETE	Allows instant option checks with immediate answer explanations.
`runQuizMode()`	COMPLETE	Executes randomized timed/untimed multi-choice quizzes.
`calculateScoreAndGrade()`	COMPLETE	Calculates total points, percentage, and letter grade performance.
`analyzeTopicPerformance()`	REFINING	Ranks top-performing vs. weak academic topics for student review.
`saveAndLoadRecords()`	REFINING	Maintains history records in external files; updating binary parsing.
`validateUserInput()`	REFINING	Sanitizes CLI buffer to prevent infinite loop errors on invalid types.

4. VALIDATION STRATEGY & CURATED TEST CASES

Real-World Test Scenarios

Key test cases selected for system validation:

- **Boundary Tests:** 100% correct, 0% score, and empty topic selections.
- **Buffer Protection:** Entering strings in place of integer choices.
- **Data Recovery:** Verifying student history persists on application restart.

Documentation & Requirements

Detailed specifications completed:

- **Requirements Doc:** Detailed memory boundaries and CLI layout specs.
- **Function Dictionary:** Documented parameters, return types, and side effects.

24	21	3	87.5%
TESTS RUN	PASSED	FAILED	PASS RATE

5. RISKS & ISSUES

- PENDING **File I/O refinement**
- PENDING **Input validation testing**
- PENDING **Cross-platform testing**

6. NEXT STEPS & DELIVERY ROADMAP

- **Finalize Remaining Refinements:** Complete output styling for topic analytics and finalize persistent file I/O auto-save synchronization routines.
- **Execute Real-Life Test Suite:** Run curated boundary and stress test cases across GCC compiler environments to ensure robust, crash-free execution.
- **Classroom Presentation & Final Delivery:** Polish inline documentation, organize project headers, and perform final demonstration.